Case Study 08: Automated Parking Lot Fee Calculation System

1. Problem Statement

Develop a Python program to calculate the parking charges for vehicles parked in a commercial lot based on vehicle category and duration of stay. The program captures the vehicle type (car or bike) and integer timestamps representing entry and exit hours within a 24-hour military clock notation (0 to 23). It must validate user entries, resolve overnight stays crossing midnight, treat matching entry and exit timestamps as a full 24-hour duration, apply hourly rates with a daily maximum price cap based on vehicle category, and print the calculated parking fee.

2. Objectives

- To validate string categories and numerical time ranges within 24-hour format boundaries ($0 \leq \text{hour} \leq 23$).
- To implement rollover time arithmetic to handle parking periods spanning across midnight.
- To utilize tuple unpacking with conditional ternary operators to configure vehicle-specific fee rates and daily caps.

- To compute final billing by clamping total hourly costs to maximum ceiling charges using Python's min() function.

3. Assumptions

- Only two vehicle types are supported: "car" and "bike" (processed case-insensitively).
- Valid hours are integers from 0 to 23 inclusive.
- If exit_time == entry, the program assumes the vehicle occupied the space for a full 24-hour parking cycle.
- If exit_time < entry, the vehicle parked overnight crossing midnight, calculated as $24 - \text{entry} + \text{exit_time}$.
- Pricing structure:
    - **Car:** Rs. 30 per hour, capped at a maximum daily ceiling of Rs. 200.
    - **Bike:** Rs. 15 per hour, capped at a maximum daily ceiling of Rs. 100.

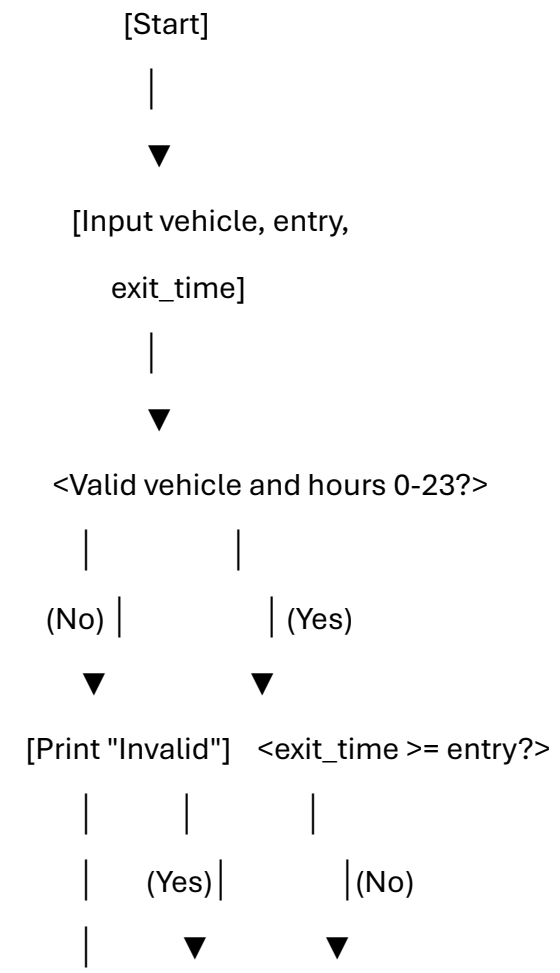
4. Algorithm

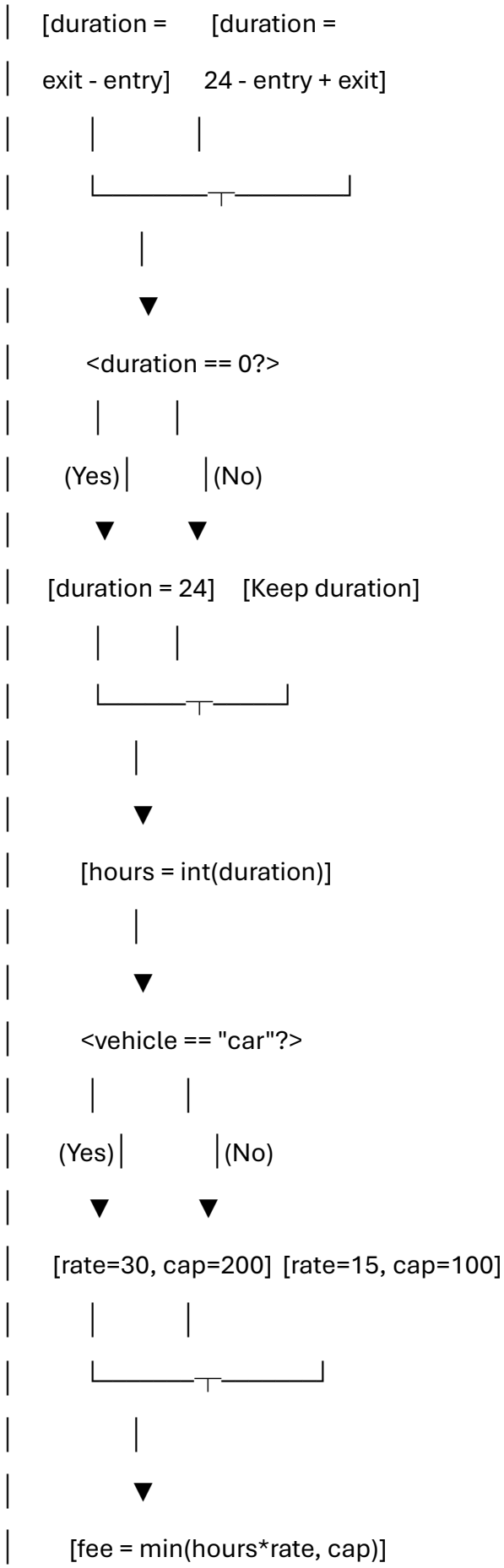
- **Step 1:** Start the program.
- **Step 2:** Prompt and read vehicle string, convert to lowercase using .lower().
- **Step 3:** Prompt and read integer values for entry and exit_time.
- **Step 4:** Validate inputs: check if vehicle not in ("car", "bike") or not (0 <= entry <= 23 and 0 <= exit_time <= 23). If invalid, print "Invalid input" and proceed to Step 10.
- **Step 5:** Compute duration:
    - If exit_time >= entry, set duration = exit_time - entry.
    - Else, set duration = 24 - entry + exit_time.
- **Step 6:** If duration == 0, reassign duration = 24.
- **Step 7:** Set hours = int(duration) if duration == int(duration) else int(duration) + 1.
- **Step 8:** Assign (rate, limit) = (30, 200) if vehicle == "car", else (15, 100).

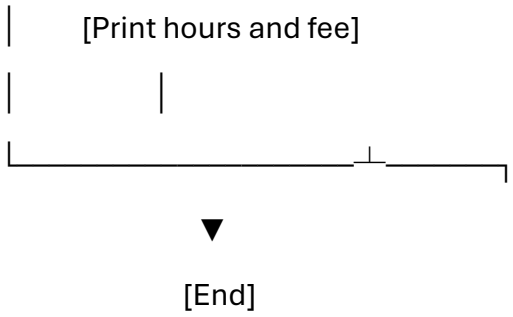
- **Step 9:** Calculate payable fee: fee = min(hours * rate, limit).
- **Step 10:** Display "Parking hours charged: <hours>" and "Parking fee: Rs. <fee>".
- **Step 11:** Stop the program.

5. Flowchart Logic / Diagram

Plaintext




[duration = exit - entry]
[duration = 24 - entry + exit]
<duration == 0?>
(Yes)
(No)
[duration = 24]
[Keep duration]
[hours = int(duration)]
<vehicle == "car"?>
(Yes)
(No)
[rate=30, cap=200]
[rate=15, cap=100]
[fee = min(hours*rate, cap)]



6. Pseudocode

Plaintext

```
BEGIN
  PRINT "Vehicle (car/bike): "
  READ vehicle_input
  SET vehicle = TO_LOWERCASE(vehicle_input)

  PRINT "Entry hour (0-23): "
  READ entry
  PRINT "Exit hour (0-23): "
  READ exit_time

  IF vehicle NOT IN ("car", "bike") OR NOT (0 <= entry <= 23 AND 0 <= exit_time <= 23) THEN
    PRINT "Invalid input"
  ELSE
    IF exit_time >= entry THEN
      SET duration = exit_time - entry
```

```
    ELSE
      SET duration = 24 - entry + exit_time
    END IF

    IF duration == 0 THEN
      SET duration = 24
    END IF

    SET hours = INTEGER(duration)

    IF vehicle == "car" THEN
      SET rate = 30
      SET limit = 200
    ELSE
      SET rate = 15
      SET limit = 100
    END IF

    SET fee = MINIMUM(hours * rate, limit)

    PRINT "Parking hours charged:", hours
    PRINT "Parking fee: Rs.", fee
  END IF
END
```

7. Python Source Code

Python

```
# Automated Parking Lot Fee Calculation System

# Read input data and normalize vehicle string
vehicle = input("Vehicle (car/bike): ").lower()
entry = int(input("Entry hour (0-23): "))
exit_time = int(input("Exit hour (0-23): "))

# Validate vehicle category and 24-hour military clock range
if vehicle not in ("car", "bike") or not (0 <= entry <= 23 and 0 <= exit_time <= 23):
  print("Invalid input")
else:
  # Compute elapsed parking duration handling overnight stays
  duration = exit_time - entry if exit_time >= entry else 24 - entry + exit_time

  # Treat identical entry/exit time as full 24-hour cycle
  if duration == 0:
    duration = 24

  # Quantize duration into billable hours
  hours = int(duration) if duration == int(duration) else int(duration) + 1

  # Configure rates and maximum daily cap by vehicle category
  rate, limit = (30, 200) if vehicle == "car" else (15, 100)
```

```
# Compute payable fee with cap protection
fee = min(hours * rate, limit)

# Display billing report
print("Parking hours charged:", hours)
print("Parking fee: Rs.", fee)
```

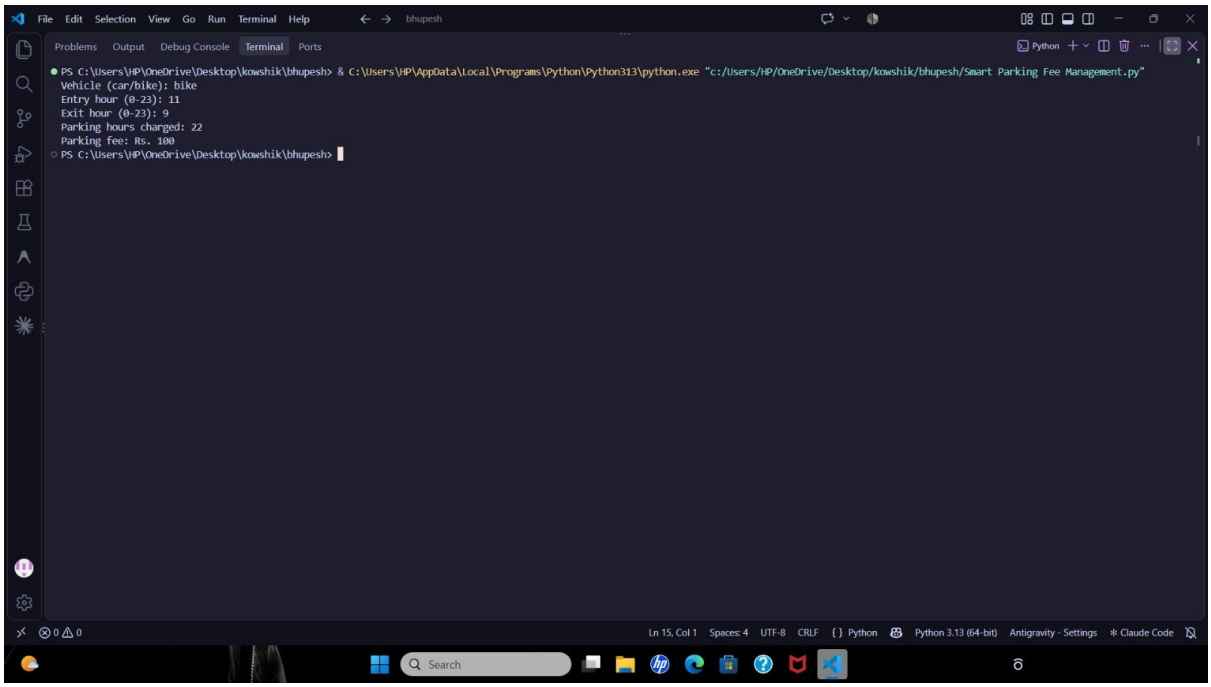
8. Test Cases

Test Case ID	Input Given	Expected Output	Actual Output	Status
TC_01 (Car Normal Hours)	Vehicle = car Entry = 9 Exit = 14	Parking hours charged: 5 Parking fee: Rs. 150	Parking hours charged: 5 Parking fee: Rs. 150	Pass
TC_02 (Car Hits Max Cap)	Vehicle = car Entry = 8	Parking hours charged: 10 Parking fee: Rs. 200	Parking hours charged: 10 Parking fee: Rs. 200	Pass

Test Case ID	Input Given	Expected Output	Actual Output	Status
	Exit = 18			
TC_03 (Bike Overnight Stay)	Vehicle = bike Entry = 22 Exit = 4	Parking hours charged: 6 Parking fee: Rs. 90	Parking hours charged: 6 Parking fee: Rs. 90	Pass
TC_04 (Full 24-Hour Cycle)	Vehicle = bike Entry = 10 Exit = 10	Parking hours charged: 24 Parking fee: Rs. 100	Parking hours charged: 24 Parking fee: Rs. 100	Pass
TC_05 (Invalid Input)	Vehicle = truck Entry = 10	Invalid input	Invalid input	Pass

Test Case ID	Input Given	Expected Output	Actual Output	Status
	Exit = 14			

9. Output Screenshots



10. Conclusion

The Automated Parking Lot Fee Calculation System was implemented and verified successfully. The script demonstrates the application of composite conditional

boundary validation, 24-hour circular modulo time math, tuple unpacking using ternary conditions, and ceiling caps using the built-in min() function.